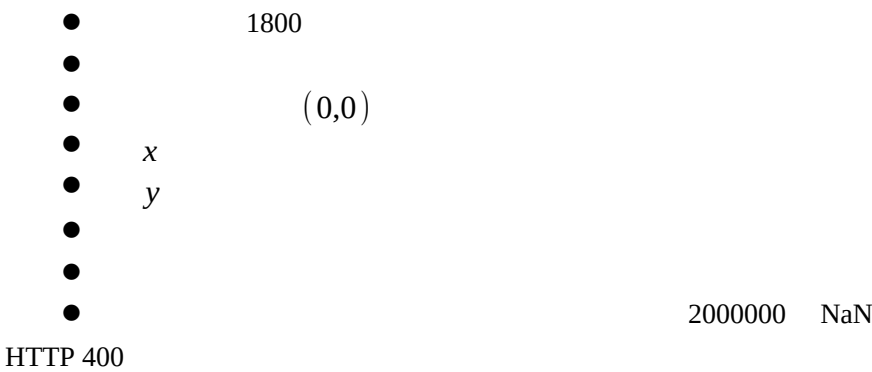


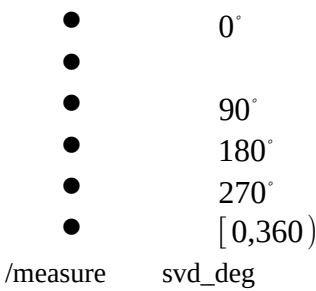
□ □ □ □ □ □ □ □ □ □ □ □ □

1. □ □ □ □

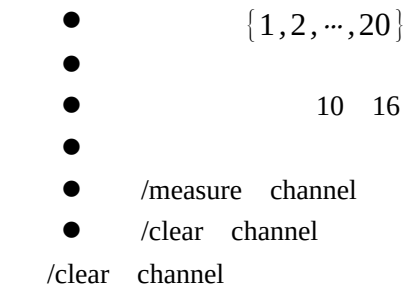
1.1. □ □ □ □ □ □ □ □



1.2. □ □ □ □



1.3. □ □



1.4. □ □ □ □



1.5. □ □ □ □

● http://127.0.0.1:2026
●
●
● 5

2. □□□□□□

2.1. □□□□□□□□□□

● 1000 1500
●
●
●

2.2. □□□□□□□□

●
● 180°

/measure

●
●
●

2.3. □□□□□□

/measure svd_deg
● svd_deg
● svd_deg
● [-1°, 1°]
●
● [0,360)
svd_deg

2.4. □□□□□□□□□□

● 5 5 /measure
measure_result="near" svd_deg
● 20 20 /clear

3. □□

4 HTTP

POST /enter

POST /measure

POST /clear

POST /exit

- /measure /clear position
- /measure channel

4. □□□□□□□□

4.1. □□□□□□□□

accepted=true

- /enter accepted=true
- /exit accepted=true
- /measure accepted=true
- /clear accepted=true
- accepted=false

accepted=false virtual_time_s 0 0

accepted=true

virtual_time_s JSON number

6

4.2. □□□□

/measure /clear

= _____

5 m/s
(0,0) (300,400) 500
 $\frac{500}{5} = 100$ ()

4.3. □□□□

/measure

= + +

- 5
- 1

/measure

- /measure channel 1
- 0
- /measure

4.4. □□□□

/clear

= +

- 3
- 5

/clear

4.5. □□□□□□□□

5 25

/enter 20 “25” “/enter 20”

/enter remaining_real_duration_s 0 1200

1200

360000

/exit

5. □□□□□□□□

5.1. □□□□

4 HTTP+JSON

- POST
- Content-Type application/json charset=utf-8 HTTP

415

- Content-Encoding identity HTTP 415
- BOM UTF-8 JSON JSON 16 65536
- /enter /measure /clear /exit
- UTF-8 JSON 65536
- channel 1 20 JSON number 1.0 1.5 HTTP

400

- JSON HTTP 200 accepted=false
- arena_id ASCII “default”
- robot_id UTF-8 1 64
- request_id UTF-8 1 128 robot_id request_id

5.2. □□□□

JSON

JSON

accepted	boolean
real_timestamp_ms	number
virtual_time_s	number
accepted=false	3

5.3. HTTP □□□□□□□□

HTTP

200	accepted=true	
200	accepted=false	JSON arena_id robot_id
400		JSON

404	HTTP				
405		POST			
409		request_id			
413		65536			
415		Content-Type	Content-Encoding		
429		/			
500					

	HTTP	JSON	accepted	real_timestamp_ms	virtual_time_s
	HTTP	accepted			
		request_id			
			ID		
robot_id	ID			HTTP 409	request_id ID arena_id
	HTTP				JSON
	" " " JSON"				
	HTTP 400	arena_id / robot_id		HTTP 200	accepted=false
request_id		request_id			

6. POST /enter

6.1.

arena_id	string	"default"
robot_id	string	
request_id	string	

```
{
  "arena_id": "default",
  "robot_id": "< >",
  "request_id": "enter-1"
}
```

6.2.

accepted	boolean	true
real_timestamp_ms	number	
virtual_time_s	number	/enter
max_virtual_duration_s	number	360000
max_real_duration_s	number	1200
remaining_real_duration_s	number	/enter

{ 0,1,2, ..., 1200 }

```
{
  "accepted": true,
  "real_timestamp_ms": 1760000000000,
  "virtual_time_s": 0,
  "max_virtual_duration_s": 360000,
  "max_real_duration_s": 1200,
  "remaining_real_duration_s": 1200
}
```

6.3.

- HTTP 200 accepted=false
 - arena_id "default"
 - robot_id
 -
 - /enter
- HTTP 400
 - JSON
 - arena_id robot_id request_id
 - robot_id request_id /
- HTTP

7. POST /measure

7.1.

arena_id	string	"default"		
robot_id	string			
request_id	string			
position	object			
position.x	number	X	JSON number	2000000
position.y	number	Y	JSON number	2000000
channel	number		{ 1,2, ..., 20 }	

```
{
  "arena_id": "default",
  "robot_id": "< >",
  "request_id": "measure-1",
  "position": { "x": 300, "y": 400 },
}
```

```
"channel": 1
}
```

7.2. □□□□

accepted=true		
accepted	boolean	true
real_timestamp_ms	number	
virtual_time_s	number	
measure_result	string	"no_signal" "near" "direction"
svd_deg	number	measure_result="direction"

7.3. □□□□□□

```
{
  "accepted": true,
  "real_timestamp_ms": 1760000000000,
  "virtual_time_s": 105,
  "measure_result": "no_signal"
}
```

```
{
  "accepted": true,
  "real_timestamp_ms": 1760000000000,
  "virtual_time_s": 105,
  "measure_result": "near"
}
```

5 svd_deg

```
{
  "accepted": true,
  "real_timestamp_ms": 1760000000000,
  "virtual_time_s": 105,
  "measure_result": "direction",
  "svd_deg": 123.45
}
```

svd_deg

7.4. □□□□□□□□□□

- HTTP 200 accepted=false
- /enter
- arena_id "default"

- robot_id
 - position
 - HTTP 400
 - JSON
 - position position.x position.y channel
 - position.x position.y NaN 2000000
 - channel 1..20
 - robot_id request_id
- HTTP

8. POST /clear

8.1.

arena_id	string	"default"			
robot_id	string				
request_id	string				
position	object				
position.x	number	X	JSON number	2000000	
position.y	number	Y	JSON number	2000000	
channel	number	{ 1,2, ..., 20 }			

```
{
  "arena_id": "default",
  "robot_id": "< >",
  "request_id": "clear-1",
  "position": { "x": 300, "y": 0},
  "channel": 3
}
```

8.2.

- 20
- position 20
- 20
- 20
- /clear

8.3.

accepted=true

accepted	boolean	true
real_timestamp_ms	number	
virtual_time_s	number	
clear_result	string	"success" "no_target_in_range"

8.4. `POST /clear`

```
{
  "accepted": true,
  "real_timestamp_ms": 1760000000000,
  "virtual_time_s": 196,
  "clear_result": "success"
}
```

- clear_result="success"
- 5

```
{
  "accepted": true,
  "real_timestamp_ms": 1760000000000,
  "virtual_time_s": 194,
  "clear_result": "no_target_in_range"
}
```

- clear_result="no_target_in_range"
- 3

8.5. `POST /measure`

/measure 7.4 /clear channel 1..20

9. POST /exit

9.1. `POST /exit`

arena_id	string	"default"
robot_id	string	
request_id	string	

```
{
  "arena_id": "default",
  "robot_id": "< >",
}
```

}

9.2. ☐ ☐

```
accepted=true
```

accepted	boolean	true
real_timestamp_ms	number	
virtual_time_s	number	/exit
exit_reason	string	

{

9.3. □ □ □ □ □ □ □ □ □

- HTTP 200 accepted=false
 - /enter
 - arena_id "default"
 - robot_id
 -
- HTTP 400
 - JSON
 - arena_id robot_id request_id
 - robot_id request_id
- HTTP /exit

10. ☐ ☐ ☐ ☐ ☐ ☐

				/					
				(m)	(s)	(s)	(s)	(s)	(s)
1	/enter	-	-	0	0	0	0	0	0
2	/measure	(300,400)	1	500	100	0	5	105	105
3	/measure	(300,400)	2	0	0	1	5	6	111

				(m)	(s)	(s)	/	(s)	(s)
							(s)		
4	/clear	(300,0)	3	400	80	0	3	83	194
5	/measure	(300,0)	2	0	0	0	5	5	199
6	/exit	-	-	0	0	0	0	0	199

- 2 (0,0) (300,400) 500 100 1
- 3 1 2 1
- 4 /clear 3 3
- 5 /measure 2 3 2

11. □□□□

4

Python

import json

from urllib.request **import** Request, urlopen

#

BASE_URL = "http://127.0.0.1:2026"

ROBOT_ID = "< >"

def post(path, payload):

```

    request = Request(
        BASE_URL + path,
        data=json.dumps(payload).encode("utf-8"),
        headers={"Content-Type": "application/json"},
        method="POST",
    )

```

with urlopen(request, timeout=5) **as** http_response:

response = json.loads(http_response.read().decode("utf-8"))

print(path, response)

return response

def base(request_id):

```

return {
    "arena_id": "default",
    "robot_id": ROBOT_ID,
}

```

```
    "request_id": request_id,  
}
```

```
def action(request_id, x, y, channel):  
    payload = base(request_id)  
    payload["position"] = {"x": x, "y": y}  
    payload["channel"] = channel  
    return payload
```

```
def main():  
    enter_response = post("/enter", base("enter-1"))  
    if enter_response.get("accepted") is not True:  
        print(" ")  
        return  
  
    remaining_time = enter_response["remaining_real_duration_s"]  
    print(" ", remaining_time, " ")  
  
    # 10  
    actions = [  
        ("/measure", action("measure-1", 300, 400, 1)),  
        ("/measure", action("measure-2", 300, 400, 2)),  
        ("/clear", action("clear-1", 300, 0, 3)),  
        ("/measure", action("measure-3", 300, 0, 2)),  
    ]
```

```
for path, payload in actions:  
    # payload request_id  
    response = post(path, payload)  
    if response.get("accepted") is not True:  
        print(" ")  
        return  
  
    if path == "/measure":  
        if response["measure_result"] == "direction":  
            print(" ", response["svd_deg"], " ")  
        elif response["measure_result"] == "near":  
            print(" ")  
        else:
```

```

        print("      ")
    else:
        if response["clear_result"] == "success":
            print("      ")
        else:
            print("      ")

    exit_response = post("/exit", base("exit-1"))
    if exit_response.get("accepted") is True:
        print("      ", exit_response["exit_reason"])

```

main()

Matlab

```

function demo_robot()

%
baseUrl = 'http://127.0.0.1:2026';
robotId = '<      >';
options = weboptions('MediaType', 'application/json', 'Timeout', 5);

enterRequest = baseRequest(robotId, 'enter-1');
enterResponse = post(baseUrl, '/enter', enterRequest, options);
if enterResponse.accepted ~= true
    fprintf('      \n');
    return;
end

remainingTime = enterResponse.remaining_real_duration_s;
fprintf('      %d \n', remainingTime);

%      10
actions = {
    '/measure', actionRequest(robotId, 'measure-1', 300, 400, 1);
    '/measure', actionRequest(robotId, 'measure-2', 300, 400, 2);
    '/clear', actionRequest(robotId, 'clear-1', 300, 0, 3);
    '/measure', actionRequest(robotId, 'measure-3', 300, 0, 2)
};

for i = 1:size(actions, 1)

```

```

path = actions{i, 1};
payload = actions{i, 2};
% payload request_id
response = post(baseUrl, path, payload, options);
if response.accepted ~= true
    fprintf(' \n');
    return;
end

if strcmp(path, '/measure')
    if strcmp(response.measure_result, 'direction')
        fprintf(' %.2f \n', response.svd_deg);
    elseif strcmp(response.measure_result, 'near')
        fprintf(' \n');
    else
        fprintf(' \n');
    end
else
    if strcmp(response.clear_result, 'success')
        fprintf(' \n');
    else
        fprintf(' \n');
    end
end
end

exitRequest = baseRequest(robotId, 'exit-1');
exitResponse = post(baseUrl, '/exit', exitRequest, options);
if exitResponse.accepted == true
    fprintf(' %s\n', exitResponse.exit_reason);
end
end

function request = baseRequest(robotId, requestId)
request = struct( ...
    'arena_id', 'default', ...
    'robot_id', robotId, ...
    'request_id', requestId);
end

function request = actionRequest(robotId, requestId, x, y, channel)

```

```

request = baseRequest(robotId, requestId);
request.position = struct('x', x, 'y', y);
request.channel = channel;
end

function response = post(baseUrl, path, payload, options)
response = webwrite([baseUrl path], payload, options);
fprintf('%s %s\n', path, jsonencode(response));
end

```

12. □□□□□□

-
- robot_id
- request_id ID
-
- HTTP accepted accepted=false virtual_time_s=0
- /enter remaining_real_duration_s 1200
- 5 5
- measure_result="direction" svd_deg
- /clear /measure
- no_signal
- /exit
- JSON direction near no_signal success no_target_in_range user_exit
-
- JSON
- 2MB